EXPERIMENT: DEPTH FIRST SEARCH (DFS)

Aim

To write a Python program to implement the Depth First Search (DFS) algorithm.

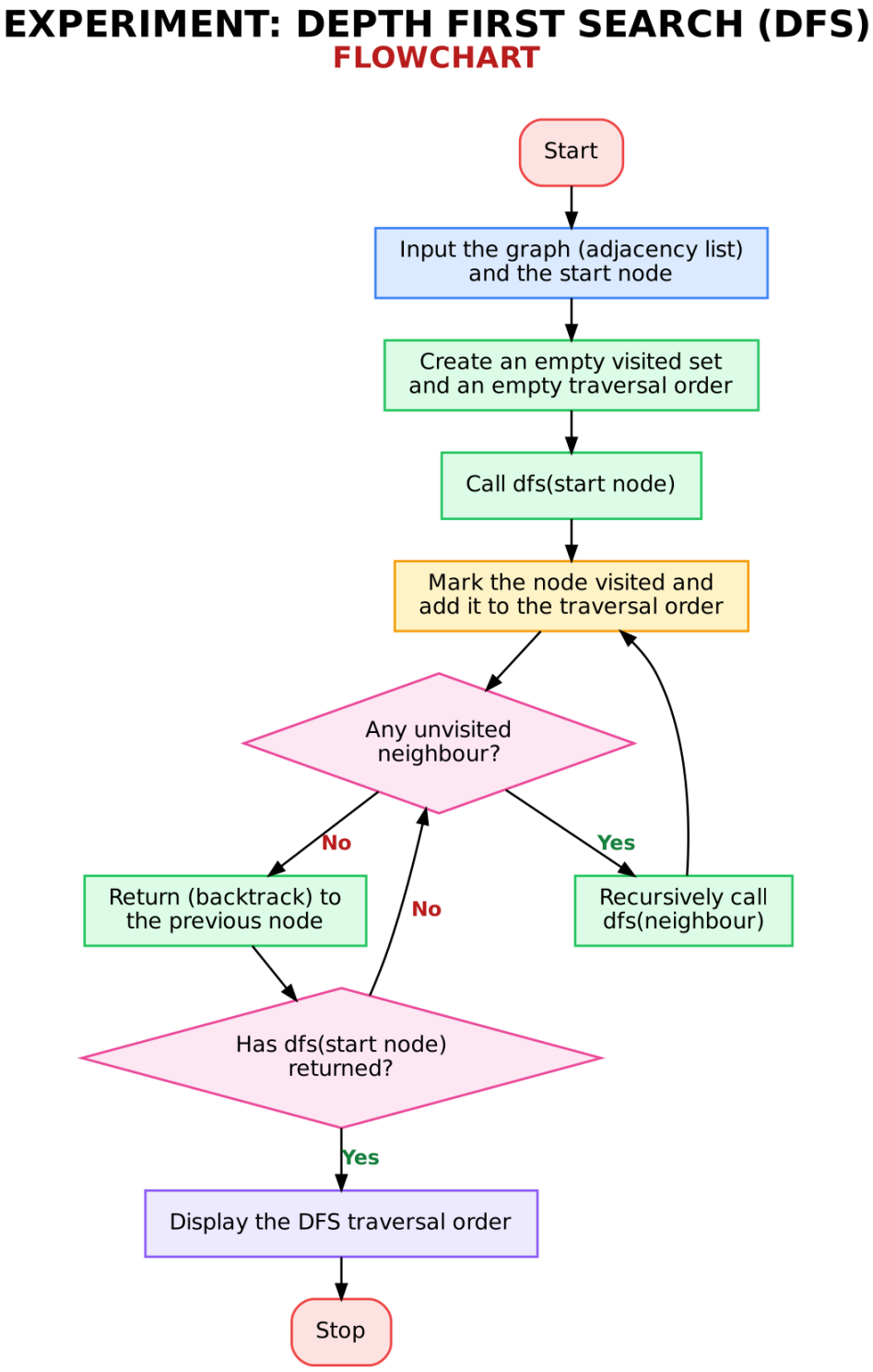
Objective

- To understand the representation of a graph using an adjacency list.
- To implement a recursive depth-wise traversal of a graph.
- To learn how DFS goes as deep as possible before backtracking.

Algorithm

1. Start.
2. Represent the graph as an adjacency list and choose the start node.
3. Create an empty visited set and an empty traversal order.
4. Call the dfs function with the start node.
5. Mark the current node as visited and add it to the traversal order.
6. For every unvisited neighbour of the current node, call the dfs function recursively.
7. When a node has no unvisited neighbours, backtrack to the previous node.
8. Display the traversal order.
9. Stop.

Flowchart:



Code (Python)

```
def dfs(graph, node, visited, order):
    visited.add(node)
    order.append(node)
    for neighbour in graph[node]:
        if neighbour not in visited:
            dfs(graph, neighbour, visited, order)
```

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

visited = set()
order = []
dfs(graph, 'A', visited, order)

print("DFS traversal starting from A:")
print(order)
```

Output

Python Shell - Output

```
DFS traversal starting from A:
['A', 'B', 'D', 'E', 'F', 'C']
```

Output Result

Thus, the Python program to implement Depth First Search was executed successfully, and the nodes were visited in the order A, B, D, E, F, C.

Conclusion

The experiment helped in understanding graph traversal. Depth First Search explores one branch completely before backtracking and uses recursion (the call stack) to remember the path.